

SGLang-Omni 每日 PR 学习日报

日期：2026-06-19 | 仓库：[sgl-project/sglang-omni](https://github.com/sgl-project/sglang-omni) | 统计窗口：过去 24 小时内 merged 的 PR

4

窗口内合并的 PR 总数

2

核心深度讲解 (Omni / 多阶段)

2

文档类 (MOSS-TTS, 简述)

25+

展开讲解的知识点

主题：可复现采样 / Seed

主题：RL Rollout 基础设施

架构：Thinker-Talker 两阶段

架构：离散 TTS 采样器

概念：Tensor Parallelism

概念：CUDA Graph

概念：logprobs / 自回归解码

编辑说明（讲解深度优先于覆盖广度）：本期把两个真正与 **Omni / 多模态 / 多阶段** 架构强相关的 PR（**#824** 统一采样种子 与 **#785 RL Rollout** 端点）讲透；另外两个是 **MOSS-TTS** 文档 PR（**#819**、**#792**），与多模态相关但属于纯文档改动，放在末尾简述，不展开。

今日目录

- #824 · feat(sampling)**：把 seed 从 API 一路打通到 model runner，统一全部 AR 采样模型的可复现性 核心
- #785 · [RL] 新增 Miles 兼容的 `/generate` rollout 端点** 核心
- #819 · [Docs] 修正 MOSS-TTS-Local 流式命令格式** 文档

1 把随机种子 `seed` 从采样 API 一路打通到 Model

Runner，统一所有自回归采样模型的可复现性

▲ 收起

MERGED

Omni / 多阶段 / 采样

#824 作者 @JingwenGu0829

[查看 PR ↗](#)

1 · 一句话概括

在 SGLang-Omni 里，把「请求带来的随机种子 `seed`」做成所有自回归 (AR) 采样模型统一遵守的契约——新增一个可复用的基类钩子，让标准文本路径、几个自定义的离散 TTS 采样器、以及 Qwen3-Omni 的 talker 全部按同一套规则用种子采样，从而让 Qwen3-Omni 的「思考(文本)+说话(音频)」整条 omni 流水线第一次做到端到端可复现。

2 · 它解决了什么问题

「可复现」是这个 PR 的全部动机。先讲清楚痛点：

- 之前 `seed` 处理是「各搞各的」(fragmented)。同一个仓库里，有的模型自己手写了一套 `seed` 逻辑，标准文本路径悄悄把 `seed` 丢掉了（用户传了 `seed` 也没用），而离散 TTS 的自定义采样器干脆完全无视 `seed`。
- 标准文本路径为什么会丢 `seed`？底层 SGLang 把「用种子采样」藏在一个叫 `enable_deterministic_inference` 的开关后面。种子虽然被填进了 `SamplingParams`，但没开那个全局开关，它就被这道「门」拦下、丢弃了。
- Qwen3-Omni 的 talker 写得「半对」。它其实是 `seed-aware` 的，但用的是 `int(seed) if seed else 0`——意思是「没给 `seed` 就固定用 0」。固定 0 会让所有未指定种子的请求都走同一条采样轨迹，这并不是真正的「随机」。

为什么非要可复现？因为强化学习 (RL) 训练强依赖它：要回放一条轨迹、要用一个 `seed` 做 **group sampling**（一组样本各自一个种子）、要做训练/

采样 **logprob** 对齐，都要求「同样的输入 + 同样的 seed → 同样的输出」。在改动前，一次完整的 Qwen3-Omni rollout (thinker 出文本 + talker 出音频) 无法复现，因为光是 thinker 这一段就把 seed 丢了。

一句话理解需求：RL 训练就像「做科学实验」，必须能把同一次实验一模一样地再跑一遍。seed 就是实验的「初始条件」。如果框架偷偷把初始条件丢掉或写死，实验就没法复现，RL 那套「采样 → 评分 → 更新」的闭环就不成立。

3 · 具体做了什么改动

改动可分成四块，核心是「抽出一个共享的 seed 工具 + 在基类装一个钩子 + 把几个特立独行的采样器接上同一套方案」。

① 新增共享工具 `sglang_omni/sampling/seed.py` (全新文件) ——所有人共用的「种子生成规则」：

`sglang_omni/sampling/seed.py` (新增)

```
# multinomial_with_seed 要求种子是一个正的 int32
SAMPLING_SEED_MASK = 0x7FFFFFFF

def new_random_sampling_seed() -> int:
    # 给「调用方没指定种子的行」临时造一个全新的随机正 int32 种子
    return int.from_bytes(os.urandom(4), "little") & SAMPLING_SEED_MASK

def derive_sampling_seed(namespace, public_seed, label=None) -> int:
    # 从「公开的请求状态」派生出一个稳定、可重复的子种子 (blake2b 哈希)
    parts = [namespace, str(public_seed)]
    if label is not None: parts.append(label)
    digest = hashlib.blake2b(":".join(parts).encode(), digest_size=8).digest()
    return int.from_bytes(digest, "little") & SAMPLING_SEED_MASK

def resolve_row_seed(public_seed) -> int:
    # 有用户种子就「掩码」成合法 int32；没有就给个全新随机种子 (保持随机)
    if public_seed is None: return new_random_sampling_seed()
    return int(public_seed) & SAMPLING_SEED_MASK
```

逐块解释：

- `SAMPLING_SEED_MASK = 0x7FFFFFFF`：这是 31 个 1 的二进制掩码（即 `int32` 的最大正数）。底层算子要求种子是正的 **32 位** 整数，所以任何来路的数字都先 `& MASK` 砍到这个范围、并去掉符号位。
- `derive_sampling_seed`：用 **blake2b** 哈希把「命名空间 + 公开种子 + 标签」搅成一个稳定的整数。关键词是「稳定」——同样的输入永远得到同样的输出，且不依赖随机数发生器的状态。下面会讲它为什么对多卡并行至关重要。
- `resolve_row_seed`：每一行（每个请求）的最终种子。有种子就规范化，没种子就真随机。

② 在基类 `ModelRunner` 装一个钩子 `_install_sampling_seeds`——这是整个 PR 的「枢纽」。它在每个解码步采样前跑一次，把每一行的种子塞进 `forward_batch.sampling_info.sampling_seed`，SGLang 一旦发现这个字段非空，就会自动路由到「带种子的采样算子」`multinomial_with_seed`：

`sglang_omni/model_runner/base.py`

```
def _install_sampling_seeds(self, forward_batch, requests):
    sampling_info = forward_batch.sampling_info
    if sampling_info.sampling_seed is not None:
        # 子类（如 Qwen3-TTS）已经自己装好了 → 不重复装
        self._validate_seeded_sampling_supported(sampling_info); return
    sampling_params = [sr.data.req.sampling_params for sr in requests]
    if all(sp.sampling_seed is None for sp in sampling_params):
        return # 整个 batch 没人带种子 → 严格 no-op, 正常服务完全不受影响
    self._validate_seeded_sampling_supported(sampling_info)
    row_seeds = []
    for row_idx, (sp, request) in enumerate(zip(sampling_params, requests)):
        seed = sp.sampling_seed
        if seed is None:
            # 混合 batch 里没带种子的行：用「请求 id 派生」的回退种子，
            # 而不是各卡各自随机 — 保证 TP 各 rank 对这一行的种子一致
            seed = _rank_shared_unseeded_sampling_seed(request, row_idx)
        elif not (0 <= seed <= SAMPLING_SEED_MASK):
            seed = resolve_row_seed(seed); sp.sampling_seed = seed # 掩
        row_seeds.append(seed)
    sampling_info.sampling_seed = torch.tensor(
        row_seeds, dtype=torch.long, device=sampling_info.device)
```

逐行讲（假设完全看不懂）：

1. 先看 `sampling_info.sampling_seed is not None`：如果种子字段已经被填了，说明某个子类（比如 Qwen3-TTS 这种「手搓派」）已经自己处理过了，基类就不抢活，直接返回。这是「子类优先、基类兜底」的设计。
2. `if all(... is None)`：如果这一批请求里一个都没带种子，直接 `return`。这就是 PR 反复强调的 **strict no-op**（严格空操作）——线上正常推理（没人要复现）时，这段代码什么都不做，零额外开销。
3. 逐行造种子：带了合法种子就用；带了「超范围」的种子就 `resolve_row_seed` 掩码并写回缓存；没带种子的行用 `_rank_shared_unseeded_sampling_seed`（基于 request id 派生）造一个回退种子。
4. 最后一行 `torch.tensor(row_seeds, ...)`：把这一批每行的种子打包成一个张量挂到 `sampling_info` 上，交给底层采样器。

那个「回退种子」为什么不直接 `os.urandom` 随机？这是本 PR 最精妙的一处。`_rank_shared_unseeded_sampling_seed` 从 request 里一层层找出 `request_id`（实在找不到就用 `row-{idx}`），再 `derive_sampling_seed` 哈希出种子。原因在「知识点」里展开讲 Tensor Parallelism。

③ 接上自定义离散 **TTS** 采样器（Higgs、Fish S2-Pro）。这两个模型原本用最朴素的 `torch.multinomial` 抽样，完全无视 seed。现在改成「带种子的行用 `multinomial_with_seed`，没种子的行原封不动走老路径」。看 Fish 的核心改动：

`sglang_omni/models/fishaudio_s2_pro/sglang_model.py`

```
# 改动前：直接 multinomial，跟种子无关
# semantic_token = top_k_indices.gather(-1, torch.multinomial(probs, 1))

# 改动后：
seeds = self._sampling_seeds[:bs]
unseeded_choice = torch.multinomial(probs, num_samples=1)
seeded_choice = multinomial_with_seed(
    torch.log(probs), seeds.clamp_min(0), self._step_count[:bs])
```

#

新路

```
choice = torch.where((seeds >= 0).unsqueeze(-1), seeded_choice, unseeded_choice)
semantic_token = top_k_indices.gather(-1, choice).squeeze(-1)
```

这里的 `torch.where(条件, A, B)` 是「逐元素三目运算符」：种子 ≥ 0 （带了种子）就取 `seeded_choice`，否则取 `unseeded_choice`。关键设计：用 `-1` 作为「没种子」的哨兵值（sentinel），保证未指定种子的解码逐字节等同于改动前（PR 用 sha256 验证过音频字节完全一致）。Higgs 的做法一样，只是 `position` 编码成 `step·N + codebook`（多了 codebook 维度，下面讲 RVQ 时解释）。

④ 把 **Qwen3-Omni talker** 统一到同一套方案。talker 因为要做 CUDA Graph 捕获，必须保留它自己静态的 `SamplingBatchInfo`（形状固定不能变），所以基类钩子不接管它——但种子的「取值规则」统一了：

```
sglang_omni/models/qwen3_omni/components/talker.py
```

```
# 改动前：sampling_seeds.append(int(seed) if seed is not None else 0)
#           ↑ 没种子就固定 0，是个 bug

# 改动后：
seed = sp.sampling_seed
if seed is None:
    # 没种子 → 用 request id 派生（每个 TP rank 算出来都一样），而不是 os.urandom
    seed = derive_sampling_seed("sglang-omni-unseeded-row", req.rid)
elif not (0 <= seed <= SAMPLING_SEED_MASK):
    seed = resolve_row_seed(seed); sp.sampling_seed = seed
sampling_seeds.append(seed)
```

受影响模型的全景（PR 自带的 **scope** 表，简化版）：

路径	模型	状态
标准 SGLang 采样器（走新基类钩子）	Qwen3-Omni thinker、Ming-Omni thinker、LLaDA2-Uni、Voxtral text-LM	修复：种子曾被 <code>enable_deterministic_inference</code> 拦下，现在直接装到 <code>sampling_info</code>
自定义离散采样器（本 PR 接线）	Higgs TTS、Fish S2-Pro	修复：改用 <code>multinomial_with_seed</code>

自定义采样器 统一到共享方案	Qwen3-Omni talker	修复：从「没种子=0」改成共享规则
本来就对（手搓，未动）	MOSS-TTS / MOSS-TTS- Local / Qwen3- TTS	不变（基类钩子检测到已装则跳过）
不在范围（连续 / flow-matching）	Voxtral acoustic、Ming- Omni talker	N/A（没有离散 token 种子）

4 · 涉及的知识点（重点展开）

① 采样 (Sampling)、temperature 与 greedy/argmax

它是什么 大模型每一步并不是直接「确定输出哪个词」，而是输出一个概率分布（每个候选 token 一个概率）。「采样」就是按这个概率掷骰子选一个。`temperature`（温度）控制骰子有多「随机」：温度高→分布被拉平→更随机有创意；温度=0→退化成永远选概率最高的那个，这叫 **greedy / argmax**，是确定性的，不需要 seed。

为什么需要 有随机性才有多样性；但有随机性就意味着「同样输入跑两次结果不同」。要复现，就得固定随机源——种子。

本 PR 里 只有 `temperature>0`（真在采样）时 seed 才有意义。PR 的验证表里专门分了「temp=0 不应有任何变化 (no-op)」和「temp>0 种子必须生效」两组实验。

② `multinomial_with_seed`：与「批次组成」解耦的有种子采样

它是什么 普通 `torch.multinomial` 依赖一个全局共享的随机数发生器 (RNG)。SGLang 的 `multinomial_with_seed(logprobs, 每行种子, 每行位置)` 不同：它把「每一行自己的种子」和「每一行当前的位置/步号」组合起来当随机源，这一行抽到什么，只取决于它自己的 (seed, position)。

为什么需要

用全局 RNG 时，第 5 行抽到什么，会受「同一批里前面 4 行抽过几次」影响。也就是说，同一个请求，跟不同的人拼在一个 batch 里，结果会变——这叫「批次组成相关」，对复现是灾难。per-row seed 让结果与批次组成无关 (**batch-composition independent**)。

本 PR 里

整套设计都建立在它之上：基类把 per-row 种子张量塞进 `sampling_info`；Higgs/Fish 把 `position` 编成「AR 步号」（Higgs 还乘上 codebook 维度），所以无论 batch 怎么拼，`seed=1234` 永远给同样的音频。

类比

就像给每个学生发一本自己专属的密码本，第 N 页对应第 N 步。不管班里坐了多少人、谁先翻、谁后翻，每个学生翻到自己第 N 页看到的内容都是固定的。

③ Model Runner / 基类钩子 / SGLang 的 `sampling_info`、`forward_batch`

它是什么

Model Runner 是 SGLang 里真正「驱动模型在 GPU 上跑一步前向 + 采样」的组件。`forward_batch` 是「这一步要算的一整批请求」的打包对象；`sampling_info`（即 `SamplingBatchInfo`）是其中专管采样参数（temperature/top_p/top_k/种子.....）的那部分。

为什么需要

把「装种子」这件事放在基类的一个统一入口（钩子）里做，所有子模型自动获得能力，避免每个模型各写一遍、写错一遍。这是典型的 **Template Method**（模板方法）/ 钩子 设计模式：基类定义流程骨架，留一个可被覆盖的「挂载点」。

本 PR 里

`_install_sampling_seeds` 在 `_sample_next_token_ids` 采样前被调用一次；子类若已自己装了种子（`sampling_seed is not None`），基类礼让跳过。

④ Tensor Parallelism (TP) 与「各 rank 必须同步」——为什么不能 `os.urandom`

它是什么

为什么需要

采样这一步，每个 **rank** 都会各自算一遍（为省一次通信）。如果某行「没种子」时各卡用 `os.urandom` 各掷各的骰子，不同卡会选出不同的 **token**——模型状态当场分裂（desync），输出直接崩坏。

本 PR 里

所以未指定种子的行，用 `derive_sampling_seed(..., request_id)` 从「所有 **rank** 都看得到的同一个 **request id**」哈希出种子——每张卡算出来必然一致，既保持了「随机感」（不同请求不同），又保证了 **rank** 间同步。这就是为什么是哈希派生而不是真随机。

类比

几个分处异地的人要「随机」却又必须选出同一个数字：不能各自扔骰子（会不一样），而要约定「拿今天的日期去算同一个哈希」——人人算得到同一个结果，但对外仍显得随机。

⑤ CUDA Graph 捕获——为什么 talker 必须保留静态 SamplingBatchInfo

它是什么

GPU 每发一个 kernel 都有「CPU 下发指令」的固定开销。**CUDA Graph** 把「一整串 GPU 操作」录制成一张图，之后一键重放，省掉反复下发的开销，对逐 token 解码这种「小而密」的场景提速明显。

为什么需要

能被「录制重放」的前提是：每次重放，输入张量的形状、地址都不能变（只能原地改数值）。所以采样信息必须是一块预先分配、形状固定的静态缓冲区。

本 PR 里

talker 用了 CUDA Graph，它的 `SamplingBatchInfo` 是静态的，不能被基类那套「动态新建张量」的钩子接管——所以本 PR 只改 talker 往那块静态缓冲里写的种子数值，不动它的结构。这解释了为什么 talker 要单独处理。

⑥ Thinker-Talker 两阶段 Omni 架构（多阶段）+ code2wav

它是什么

Qwen3-Omni 把「能听会说」拆成两个自回归阶段：**thinker** 负责理解输入、生成文本（思考）；**talker** 接着把文本变成音频离散 **token**（说话）；最后 **code2wav** 把音频 token 还原成波形。PR 里甚至把它们分到不同 GPU（thinker GPU0 /

talker+code2wav GPU1)，即 **disaggregated**（分离式部署）。

为什么需要

「理解+生文本」和「文本→语音」是两类不同的计算，拆开能各自优化、各自扩容。但两段都各自采样，意味着要复现「一句完整的语音回答」，必须两段都吃同一套种子规则。

本 PR 里

之前 thinker 把种子丢了，所以即便 talker 接住种子，整条链也不可复现。本 PR 让 thinker（走基类钩子）和 talker（统一取值规则）种子一致，于是「thinker 文本 + talker 音频」端到端可复现——验证里同一个 `seed=1234` 跑两次音频哈希完全相同。

⑦ 离散 TTS、Codebook / RVQ、AR step、delay pattern

它是什么

现代 TTS 常把音频压成一串离散整数 **token**（像「音频的字」）。**RVQ**（残差矢量量化）用多个 **codebook** 层层逼近：第 1 本管粗轮廓，后面几本管细节残差。每个解码步（**AR step**）要为多个 **codebook** 各抽一个 **token**。Higgs/MOSS 这类还有 **delay pattern**（不同 codebook 错开几步生成）。

为什么需要

把连续音频离散化后，就能像「预测下一个词」一样用自回归 Transformer 生成语音；多 codebook 是为了在有限词表下表达高保真音频。

本 PR 里

Higgs 给每次抽样的 `position = step·N + codebook`（N 是 codebook 个数）——这样同一步里不同 **codebook**、不同步之间的随机源都各不相同且可复现。Fish 的 `position` 就是 `step_count`。代码里还维护 `step_count` 随「活跃行」自增，并在 `reset_row` 时清零，避免复用脏状态。

⑧ 工程细节：哨兵值 (sentinel)、no-op、严格字段访问、掩码

它是什么

哨兵值：用一个不可能的正常值（这里是 `-1`）表示「没有」。**no-op**：不产生任何效果的操作。掩码：用按位与把数字砍进合法范围。

为什么需要

用 `-1` 哨兵区分「带种子/未带种子」，能让未带种子的行逐字节走老路径，从而保证「不破坏现有行为」这条铁律可被字节级验证。严格字段访问（不用防御性 `getattr` 兜默认）则让

「契约字段缺失」时立刻大声报错，而不是悄悄跑出错误结果。

本 PR 里

Higgs/Fish 的 `NO_SEED=-1`、基类的 `all(... is None) → return`、talker 取消旧的 `getattr` 改严格访问，都是这套思路。PR 还专门加了单测验证「未种子时 no-op」「混合 batch 各行不串味」。

5 · 我能学到什么 / 延伸思考

- 「把横切关注点上提到基类」：seed 是个典型的横切关注点（cross-cutting concern），到处都要、又容易各写各错。抽成共享工具 + 基类钩子，是消除重复、统一契约的范本。
- 「正确性的可验证性」优先于「优雅」：用 `-1` 哨兵保留老路径，看似不漂亮，却换来「未种子行逐字节等同改动前」这种可用 **sha256** 客观验证的强保证。改动越底层，越要让「无变化」可被证明。
- 分布式确定性的核心矛盾：「随机」和「多卡一致」天然冲突，解法是用共享的确定性输入哈希出伪随机。这套思路在任何多 worker / 多 rank 系统里都通用。
- 前置知识建议：(1) 自回归解码与采样（temperature/top-k/top-p）；(2) PyTorch 张量与 `torch.where` 向量化写法；(3) Tensor Parallelism 与集合通信 all-reduce；(4) CUDA Graph 的「静态形状」约束；(5) 神经音频编解码器（EnCodec / RVQ）。

2 新增 Miles 兼容的 `/generate` RL-rollout 端点，把 rollout 产物端到端打通

▲ 收起

MERGED

RL / Omni / 推理服务

#785

作者 @yxs

[查看 PR ↗](#)

1 · 一句话概括

给推理服务和 router 加一个 `POST /generate` 端点，长得跟 RL 训练框架 **Miles** 约定的格式一致：训练器一次调用就能驱动一次生成，并原样读回它做

策略更新所需要的东西——每个采样 token 的 `logprob`、`weight_version` 等，同时支持「直接喂预先 tokenize 好的 `input_ids`」。

2 · 它解决了什么问题

这是为 **RL**（强化学习）训练 **Qwen3-Omni** 铺基础设施。RL 的训练循环是：用当前策略生成一批回答（**rollout**）→ 打分 → 用这些数据更新策略。要让外部 RL 训练器（Miles）能把 SGLang-Omni 当「生成引擎」用，必须解决几个老代码的缺口：

- 没有 **Miles** 要的端点格式。Miles 期望一个 `/generate` 接口，请求里直接给 `input_ids`，响应里回 `meta_info.output_token_logprobs`。老服务没有这个契约。
- 预先 **tokenize** 的 `input_ids` 进不去。Qwen3-Omni 的预处理之前只认 **messages**（聊天消息列表），你要是直接塞 token id，它会在 `normalize_messages` 那里 500 报错。
- 为什么必须能直接喂 **token id**？因为 Miles 训练时用的是它自己 **tokenize** 出来的那串 **token**。如果生成端在服务器再 tokenize 一遍，两边的 token 可能对不齐（**drift**）——而 RL 要求「训练用的 token」和「采样用的 token」必须严格一致，否则梯度算错。
- **logprob** 不能事后从 **logits** 重算。策略更新需要的是「当时那个策略真正采样这个 token 时的 **logprob**」，它带着 temperature 等语义。事后拿原始 logits 重算会得到不一样的值，破坏 on-policy 假设。

3 · 具体做了什么改动

改动面较广（serve / client / model_runner / preprocessor / router 都碰了），抓四条主线：

① 在 `openai_api.py` 注册 `/generate` 端点 + 请求/响应契约。请求支持三选一的 prompt 形式（`input_ids` / `messages` / `prompt`）加采样与 rollout 开关；响应是 `text + meta_info`。核心是把内部结果搬进 Miles 约定的字段，并做严格的一致性校验：

`sglang_omni/serve/openai_api.py`（`_build_generate_response` 摘录）

```

completion_tokens = usage.completion_tokens if usage else 0 # 永远是 int
finish_reason = GenerateFinishReason(
    type=finish_type,
    length=completion_tokens if finish_type == "length" else None)

# 说要 logprob 却没拿到 → 直接 500, 绝不静默返回错误数据
if req.return_logprob and result.output_token_logprobs is None:
    raise HTTPException(500, "backend did not return output_token_logprobs")
# logprob 条数必须 == 生成 token 数, 否则也是 500
if req.return_logprob and len(result.output_token_logprobs) != completion_tokens:
    raise HTTPException(500, "... length mismatch ...")

meta_info = GenerateMetaInfo(
    finish_reason=finish_reason,
    prompt_tokens=prompt_tokens, completion_tokens=completion_tokens,
    weight_version=result.weight_version,
    request_metadata=req.metadata, # 原样回显训练器传来的 metadata
    output_token_logprobs=result.output_token_logprobs if req.return_logprob else None,
    omni_rollout=result.omni_rollout if req.return_omni_rollout else None)

```

注意两个工程取舍：计数字段永远序列化成 **int**（未知时为 **0**），绝不 **为 null**——因为 RL 端会累加（`cached_tokens += ...`），碰上 `None` 会抛 `int + None`。而 `output_token_logprobs` 是 `[logprob, token_id]` 一项一个 token，刻意对齐 SGLang 原生 `meta_info` 格式。

② 在基类 `model_runner` 里「从采样器记录 **logprob**」。这是正确性的关键：不重算，直接抄采样器算好的值。

sglang_omni/model_runner/base.py

```

wants_rollout_logprob = any(sr.data.return_logprob for sr in requests)
if wants_rollout_logprob:
    self._enable_sampler_logprobs(forward_batch, len(requests)) # 打开采样器
next_token_ids = self.tp_worker.model_runner.sample(logits_output, forward_batch)
if wants_rollout_logprob:
    next_token_logprobs = logits_output.next_token_logprobs # 采样器算好的
    if next_token_logprobs is None:
        raise RuntimeError("Sampler did not populate next_token_logprobs")
    self._record_rollout_logprobs(next_token_logprobs, next_token_ids,

```

`_record_rollout_logprobs` 再把每行的 `[logprob, token_id]` 追加到该请求的列表里，并断言三者长度一致（`logprobs == token_ids ==`

`requests`)，不一致立刻报「batch-size mismatch」。配套的 `sampled_logprobs_to_list` (在 `scheduling/types.py`) 负责把采样器输出的张量稳妥转成 `list[float]`，它的 docstring 明说：「采样器拥有 logprob 的语义 (temperature、original-logprob 模式)，rollout 代码应保留这些值而不是从 logits 重算」。

③ 给 Qwen3-Omni 预处理加「预 tokenize 分支」。识别到输入是一串纯 int (token id) 就绕开 chat 模板 + HF processor，直接构造 thinker 状态：

`sglang_omni/models/qwen3_omni/components/preprocessor.py`

```
def _is_pretokenized_prompt(inputs):
    # 是「非空 list 且每个元素都是 int」→ 判定为预 tokenize 的 prompt
    return isinstance(inputs, list) and bool(inputs) and all(isinstance

def _preprocess_pretokenized(self, payload, token_ids):
    input_ids = torch.tensor(token_ids, dtype=torch.long)
    attention_mask = torch.ones_like(input_ids)
    validate_prompt_seq_len(input_ids, ...) # 仍做长度校验
    return self._finalize_state(
        payload, input_ids=input_ids, attention_mask=attention_mask,
        prompt_text="", full_mm_inputs={},
        encoder_inputs={"image_encoder":{"_skip":True,"_result":{}}},
                        "audio_encoder":{"_skip":True,"_result":{}}})
```

顺手做了一次小重构：把「组装 thinker 状态」的逻辑抽成

`_finalize_state`，让正常 messages 路径和新预 tokenize 路径 共用同一处状态组装代码 (single source of shape)，避免两条路径未来跑偏。注意：多模态 (带图/音) 仍走 messages 路径，预 tokenize 分支只处理纯文本。

④ client 与 router 端到端透传 rollout 产物。client 在聚合流式结果时新增对 `output_token_logprobs / omni_rollout / weight_version` 的收集，并特别保留「空列表 `[]`」与「`null`」的区别——零生成 token 是合法值 (空列表)，不能和「没捕获到 logprob」(null) 混为一谈。router 则做不透明转发 (opaque forwarding)：原样把 metadata 等带过去再带回来。

留了一个「半成品」是有意为之：`omni_rollout` 字段已经端到端「布好线」，但本 PR 暂不填充 (forward-declared only)——多模态的「可训练动作

容器」是下一步。Miles 这边先传 `return_omni_rollout=false`。这是「先打通管道、再灌内容」的渐进式交付。

4 · 涉及的知识点（重点展开）

① RL Rollout / 策略更新 / on-policy

它是什么 **Rollout** = 让当前模型（策略 policy）实际生成一批回答的过程。RL 训练循环 = **rollout** 生成 → 用奖励打分 → 更新策略，再循环。**on-policy** 指「用来更新的数据，必须来自当前这个策略」。

为什么需要 更新策略时要计算「这个 token 在当前策略下的概率」相对「采样时的概率」的比值（如 PPO 的重要性采样比）。所以必须把采样当时的 **logprob** 准确记下来。

本 PR 里 整个端点就是为「让 Miles 驱动 rollout 并读回 logprob」服务；坚持「logprob 来自采样器、不重算」正是为守住 on-policy 的正确性。

② logprob（对数概率）与「为什么不能重算」

它是什么 模型对某个 token 给出的概率取对数。概率越接近 1，logprob 越接近 0；越小则越负。所以 PR 校验里「所有 $\text{logprob} \leq 0$ 」是合理性检查。

为什么需要 采样器算的 logprob 已经包含了 temperature、top-p 截断、original-logprob 模式等语义。事后拿干净 logits 重算，得到的是「另一个分布」的 logprob，与真实采样分布不符。

本 PR 里 `_enable_sampler_logprobs` 打开开关让采样器顺带吐出 `next_token_logprobs`，`sampled_logprobs_to_list` 原样搬运。验证里确认 `len(logprobs)==completion_tokens` 且能 decode 回各自的文本（不串行）。

③ Tokenizer / chat 模板 / token drift

它是什么

(role+content) 拼成模型实际看到的字符串的规则（含特殊标记）。同一段话，模板/版本/特殊符号略有差异，切出的 token 序列就可能不同——这种偏差叫 **drift**。

为什么需要

RL 训练器和推理引擎若各自 tokenize，token 序列一旦 drift，训练计算梯度用的序列就和实际采样的序列对不上，训练直接错。

本 PR 里

新增预 tokenize 分支，让 Miles 把「它训练用的那串 token」原样喂进 thinker，绕过 **chat** 模板和 **HF processor**，从根上消除 drift。验证：同一 prompt 分别以 `input_ids` 和 `messages` 送入，贪心解码得到完全相同的 token 序列。

④ HF processor / 多模态编码器 (image/audio encoder) 与 thinker 状态

它是什么

HF processor 是 HuggingFace 的多模态预处理器：把图片/音频转成张量特征，再交给对应的 **encoder**（视觉/音频编码器）抽成 embedding，和文本 token 一起喂给 thinker。**thinker** 状态就是「这次推理 thinker 需要的全部输入」的打包。

为什么需要

Omni 模型要同时吃文本/图/音，所以预处理阶段要把各模态分别编码好。但纯文本 rollout 用不到图音编码器。

本 PR 里

预 tokenize 分支把 `image_encoder / audio_encoder` 标成 `_skip:True`（跳过），只构造文本 thinker 状态——既省算力，又精确表达「这是纯文本 rollout」。多模态输入仍回到完整 `messages` 路径。

⑤ 批次对齐 / continuous batching (不串味)

它是什么

推理引擎为了吞吐，会把多个不同请求塞进同一个 **batch** 同时算（SGLang 的 continuous batching 还会动态加入/移除请求）。「对齐」指每一行的输出、logprob 必须严格归属到它自己那个请求。

为什么需要

一旦行错位，A 请求会拿到 B 请求的 logprob/token，RL 数据被污染却毫无报错——是最危险的隐形 bug。

本 PR 里

⑥ weight_version 与 router 的不透明转发

它是什么 `weight_version` 标记「这次生成用的是哪一版模型权重」。RL 里权重不停更新，必须知道某条 rollout 数据是哪版策略产出的。不透明转发 = router 不解析、不改动某些字段（如 metadata），原样转给后端再原样带回。

为什么需要 训练器要据 `weight_version` 判断数据新鲜度（off-policy 程度）；metadata（如 `rollout_id`）是训练器自己的簿记，router 不该乱碰。

本 PR 里 `route_metadata.py` / `app.py` 把这些字段沿链路透传；响应里 `request_metadata` 原样回显，`weight_version` 一路带回 `meta_info`。

⑦ 渐进式交付：forward-declared 字段

它是什么 先把一个字段的「管道」（请求→调度→响应的传递路径）全部布好，但暂时不产生真实内容。等下一个 PR 再填。

为什么需要 大功能拆小步走：先让接口契约稳定、让上下游能联调，降低一次性大改的风险与 review 难度。

本 PR 里 `omni_rollout`（多模态可训练动作容器）就是这样——字段、透传、校验都在了，但本期不填充，Miles 先传 `return_omni_rollout=false`。

5 · 我能学到什么 / 延伸思考

- 「契约即正确性」：这个 PR 一大半工作不是算法，而是把字段类型、长度、空值语义全部钉死（int 永不为 null、空列表≠null、logprob 条数必须等于 token 数、不符就 500）。给别的系统当依赖时，严格契约比「能跑」重要得多。
- 「在数据产生的源头取真值」：logprob 一定要在采样器那一刻记录，而不是事后重算。任何「事后重建」都可能引入与真实过程的偏差——这是可观测性/数据管线的通用原则。

- 消除重复以防漂移：把两条预处理路径收敛到 `_finalize_state` 一处，是「single source of truth」的小而美实践。
- 前置知识建议：(1) PPO / GRPO 等 RL 算法里 logprob、重要性采样比的作用；(2) OpenAI 风格 HTTP 推理 API 与 SSE 流式协议；(3) tokenizer / chat template 的细节；(4) SGLang 的 scheduler、continuous batching、router 架构；(5) Qwen3-Omni 的 thinker-talker 多阶段流水线。

3 [Docs] 修正 MOSS-TTS-Local 流式命令格式

MERGED

文档

#819 作者 @SandyLuXY

▲ 收起

[查看 PR ↗](#)

一句话：修文档——MOSS-TTS-Local 的流式示例把响应直接管道给 `ffmpeg -f s16le` 时，必须显式加上 `"stream_format":"audio"`，否则会输出「噪声样」的 WAV 并触发 ffmpeg 的 PCM 解码错误（即命令退出码是 0）。

解决的问题：原 cookbook 漏了这个参数。作者实测两次「不加」会得到 `mean_abs≈0.59` 的噪声样输出、ffmpeg 报 `Invalid PCM packet`；「加上」后 `mean_abs≈0.06` 的正常语音、无解码错误。改动只动 `docs/cookbook/moss_tts_local.md`：流式段要求 `"stream":true` + `"response_format":"pcm"` + `"stream_format":"audio"`，并在参数表里补充 `stream_format` 说明。

顺带的知识点：**PCM**（脉冲编码调制）是最原始的未压缩音频字节流；`s16le` = 16-bit 小端有符号整数。流式 TTS 直接吐裸 PCM 字节，需要告诉服务端「按音频流封装」（`stream_format:audio`），下游 ffmpeg 才能正确把字节解析成音频帧，否则把控制/封装信息当成样本，就成了噪声。

4 docs：新增 MOSS-TTS-Local cookbook

MERGED

文档

#792 作者 @MelodyyyYin

▲ 收起

[查看 PR ↗](#)

一句话：为 `MOSS-TTS-Local-Transformer-v1.5` 新增一份面向用户的 cookbook 文档，并注册到文档目录树。纯文档，一个新文件 + 一行 toctree。

内容要点（也是几个有用的领域概念）：

- **frame-local decoding**（无 **delay pattern**）：与普通 MOSS-TTS 不同，MOSS-TTS-Local 在每一帧内部解码，不用「错位延迟」的 codebook 生成方式——结构更简单。（对照 #824 里讲的 delay pattern。）
- **RVQ-12** 音频码本：12 层残差矢量量化（接 #824 里的 RVQ 概念，12 本 codebook）。
- **48kHz** 立体声输出、并发 **16** 时 **RTF<1**：**RTF**（Real-Time Factor，实时率）= 合成耗时/音频时长，<1 表示比实时还快，能跟得上流式播放。
- 部署：checkpoint 能自动解析出 `MossTTSLocalModel`，所以 `serve` 只需 `--model-path`（像 Higgs TTS 一样无需 `--config`）；默认 AR 引擎与 codec/vocoder 分到两张卡，故 `CUDA_VISIBLE_DEVICES=0,1`。

工程态度可借鉴：作者强调「每条事实都对照源码核验」，并主动删掉了仓库里无据可查的「**12.5 Hz** 帧率」说法——文档以可验证为准，不臆造数字。

本日报由自动化例程于 2026-06-19 生成 · 数据源：GitHub `sgl-project/sglang-omni` · 统计口径：
过去 24 小时窗口内 merged 的 PR（共 4 个，深度讲解 2 个核心、简述 2 个文档）。
讲解基于各 PR 的实际 diff 与描述撰写，代码片段为便于阅读做了精简/高亮处理，完整代码以仓库为准。